



PILHAS E FILAS EM JAVASCRIPT

ESTRUTURA DE DADOS

CST em Desenvolvimento de Software Multiplataforma



PROF. Me. TIAGO A. SILVA



LIVRO DE REFERÊNCIA DA DISCIPLINA

- **BIBLIOGRAFIA BÁSICA:**

- GRONER, Loiane. **Estrutura de dados e algoritmos com JavaScript**: escreva um código JavaScript complexo e eficaz usando a mais recente ECMAScript. **São Paulo: Novatec Editora, 2019.**

- **NESTA AULA:**

- **Capítulos 4 e 5**



PARA SOBREVIVER AO JAVASCRIPT

Non-zero value



null



0



undefined



IMPLEMENTAÇÃO DA ESTRUTURA DE DADOS PILHA

Atenção as regras de acesso aos elementos

IMPLEMENTAÇÃO DE UMA PILHA

- Estrutura interna (**items**): Usamos um objeto para armazenar os elementos da pilha, com chaves representando as posições.
 - **adicionar(elemento)**: Insere o elemento no topo da pilha, na posição correspondente ao tamanho atual. Em seguida, incrementa o tamanho da pilha.
 - **remover()**: Remove e retorna o elemento do topo da pilha, ajustando o tamanho após a remoção. Verifica se a pilha está vazia antes de tentar remover.
 - **topo()**: Retorna o elemento no topo da pilha sem removê-lo.
 - **estaVazia()**: Verifica se a pilha está vazia, retornando **true** ou **false**.
 - **tamanhoPilha()**: Retorna o número de elementos na pilha.
 - **limpar()**: Remove todos os elementos da pilha, redefinindo os valores de **items** e **tamanho**.

IMPLEMENTAÇÃO DE UMA PILHA

- Essa implementação simula o comportamento básico de uma pilha, **sem recorrer a métodos** nativos como **push()** e **pop()**, e realiza o gerenciamento de dados e índices de maneira manual.



REGRAS DA PILHA

FILO
First in – Last out

*Primeiro que
entra*

*Último que
saí*

LIFO
Last in – Last out

*Último que
entra*

*Primeiro que
saí*



ADICIONANDO ELEMENTOS À PILHA

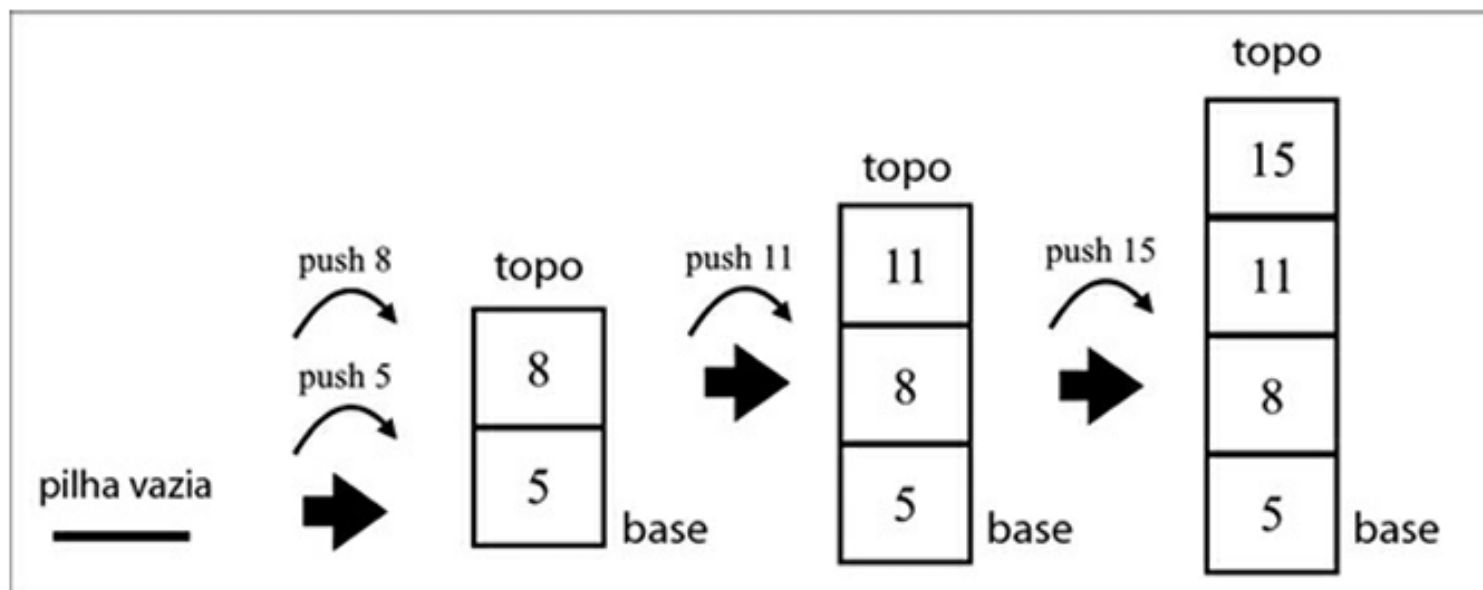


Figura 4.5



REMOVENDO ELEMENTOS DA PILHA

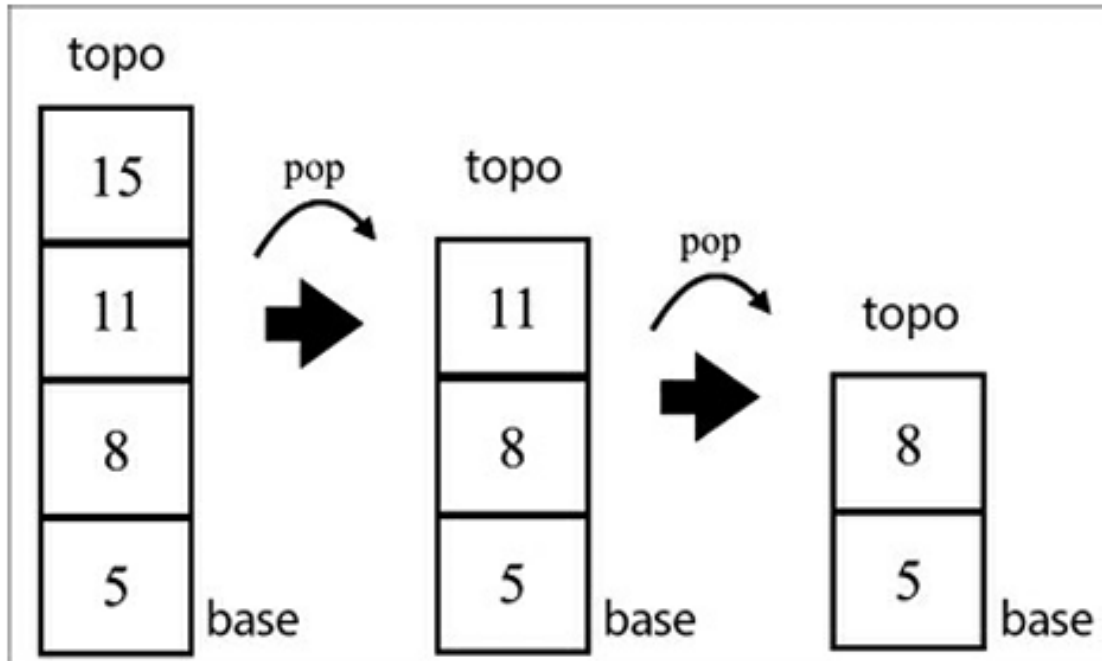


Figura 4.6



IMPLEMENTAÇÃO DO PROTÓTIPO DA CLASSE

Aula 04 - Pilhas - Pilha_Prototipo.js

```
1  class MinhaPilha {  
2  
3      #items = [];  
4      #tamanho = 0;  
5  
6      adicionar(elemento) { }  
7      remover() { }  
8      topo() { }  
9      limpar() { }  
10     estaVazia() { }  
11     tamanhoPilha() { }  
12 }  
13  
14 module.exports = MinhaPilha;
```



IMPLEMENTAÇÃO DOS ATRIBUTOS DA CLASSE

Aula 04 - Pilhas - Pilha.js

```
1 class MinhaPilha {  
2  
3     // Usamos uma lista para armazenar os elementos da pilha  
4     #items = [];  
5  
6     // Mantemos o controle do tamanho da pilha  
7     #tamanho = 0;
```



IMPLEMENTAÇÃO DO MÉTODO adicionar(elemento)



Aula 04 - Pilhas - Pilha.js

```
9  // Adiciona um elemento ao topo da pilha
10 adicionar(elemento) {
11
12     // Insere o elemento na posição atual do tamanho
13     this.#items[this.#tamanho] = elemento;
14
15     // Incrementa o tamanho
16     this.#tamanho++;
17 }
```



IMPLEMENTAÇÃO DO MÉTODO `remove()`

```
Aula 04 - Pilhas - Pilha.js

19 // Remove e retorna o elemento do topo da pilha
20 remove() {
21
22     // Se a pilha estiver vazia, retorna undefined
23     if (this.#tamanho === 0) {
24         return undefined;
25     }
26
27     // Pega o item no topo da pilha
28     const ultimoItem = this.#items[this.#tamanho - 1];
29
30     // Remove o item do topo
31     delete this.#items[this.#tamanho - 1];
32
33     // Decrementa o tamanho da pilha
34     this.#tamanho--;
35
36     // Retorna o item removido
37     return ultimoItem;
38 }
```



IMPLEMENTAÇÃO DO MÉTODO topo()

Aula 04 - Pilhas - Pilha.js

```
40 // Retorna o elemento no topo da pilha sem removê-lo
41 topo() {
42
43     // Se a pilha estiver vazia, retorna undefined
44     if (this.#tamanho === 0) {
45         return undefined;
46     }
47
48     // Retorna o item no topo
49     return this.#items[this.#tamanho - 1];
50 }
```



IMPLEMENTAÇÃO DO MÉTODO `limpar()`

● ● ● Aula 04 - Pilhas - Pilha.js

```
59 // Limpa a pilha
60 limpar() {
61     // Reseta os itens
62     this.#items = [];
63
64     // Reinicializa o tamanho
65     this.#tamanho = 0;
66 }
```



IMPLEMENTAÇÃO DOS MÉTODOS `estaVazia()` E `tamanhoPilha()`



Aula 04 - Pilhas - Pilha.js

```
52 // Verifica se a pilha está vazia
53 // Verifica se o tamanho da pilha é zero
54 estaVazia = () => this.#tamanho === 0;
55
56 // Retorna o número de elementos na pilha
57 tamanhoPilha = () => this.#tamanho;
```



COMO USAR A CLASSE?

```
Aula 04 - Pilhas - app.js

1  const MinhaPilha = require('./Pilha.js');
2
3  // Exemplo de uso
4  const minha_variavel = new MinhaPilha();
5
6  minha_variavel.adicionar("a");
7  minha_variavel.adicionar(20);
8  minha_variavel.adicionar(30);
9
10 // Saída: 30 (Elemento no topo)
11 console.log(minha_variavel.topo());
12
13 // Saída: 30 (Remove o elemento do topo)
14 console.log(minha_variavel.remover());
15
16 // Saída: 20 (Agora o topo é 20)
17 console.log(minha_variavel.topo());
18
19 // Saída: 2 (Dois elementos restantes)
20 console.log(minha_variavel.tamanhoPilha());
```



IMPLEMENTAÇÃO DA ESTRUTURA DE DADOS FILA

Atenção as regras de acesso aos elementos

IMPLEMENTAÇÃO DE UMA FILA

- Estrutura interna (**items**): Usamos um objeto para armazenar os elementos da fila. As chaves do objeto representam as posições dos elementos na fila.
 - **enqueue(elemento)**: Adiciona um novo elemento ao final da fila. O índice do fim da fila (**this.fim**) é incrementado cada vez que um novo elemento é inserido.
 - **dequeue()**: Remove o primeiro elemento da fila. O índice do início (**this.inicio**) é incrementado após a remoção de um elemento, e o item correspondente é excluído do objeto.
 - **front()**: Retorna o elemento na frente da fila sem removê-lo.
 - **isEmpty()**: Verifica se a fila está vazia, comparando os índices de início e fim.
 - **size()**: Calcula o número de elementos na fila subtraindo o índice do início do índice do fim.
 - **clear()**: Limpa completamente a fila, reiniciando os valores.

IMPLEMENTAÇÃO DE UMA FILA

- Essa implementação manual simula o comportamento de uma fila usando apenas um objeto como armazenamento interno e gerenciando manualmente os índices para garantir o funcionamento correto da fila sem o uso de funções nativas como **push()** e **shift()**.



REGRAS DA FILA

FIFO
First in – First out

*Primeiro que
entra*

*Primeiro que
saí*

LIFO
Last in – Last out

*Último que
entra*

*Último que
saí*



ADICIONANDO ELEMENTOS À FILA (ENQUEUE)

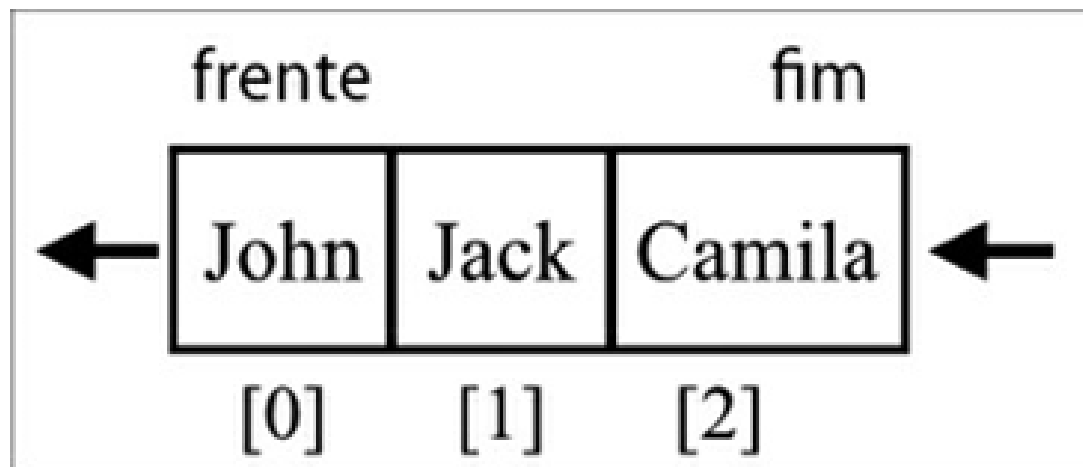


Figura 5.2



ADICIONANDO ELEMENTOS DA FILA (DEQUEUE)

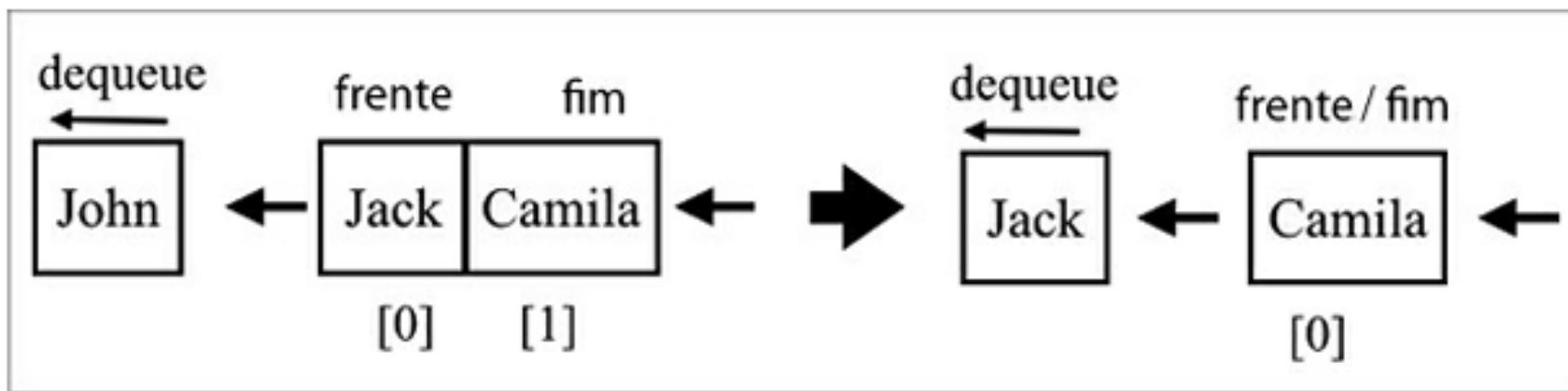


Figura 5.3



IMPLEMENTAÇÃO DO PROTÓTIPO DA CLASSE

● ● ● Aula 05 - Filas - Fila_prototipo.js

```
1  class Fila
2  {
3      #items = [];
4      #inicio = 0;
5      #fim = 0;
6
7      enqueue(elemento) { }
8      dequeue() { }
9      front() { }
10     estaVazia() { }
11     tamanho() { }
12     limpar() { }
13 }
14
15 module.exports = Fila;
```



IMPLEMENTAÇÃO DOS ATRIBUTOS DA CLASSE

Aula 05 - Filas - Fila.js

```
1  class Fila
2  {
3      // Usamos uma lista para armazenar os itens
4      #items = [];
5
6      // Representa o índice do início da fila
7      #inicio = 0;
8
9      // Representa o índice do fim da fila
10     #fim = 0;
```



IMPLEMENTAÇÃO MÉTODO ENQUEUE

Aula 05 - Filas - Fila.js

```
12 // Adiciona um elemento ao final da fila (enqueue)
13 enqueue(elemento)
14 {
15     // Coloca o elemento no fim da fila
16     this.items[this.#fim] = elemento;
17
18     // Incrementa o índice do fim da fila
19     this.#fim++;
20 }
```



IMPLEMENTAÇÃO MÉTODO dequeue()

```
Aula 05 - Filas - Filajs

22 // Remove e retorna o primeiro elemento da fila (dequeue)
23 dequeue()
24 {
25     // Se a fila estiver vazia, retorna undefined
26     if (this.estaVazia()) {
27         return undefined;
28     }
29
30     // Obtém o primeiro elemento
31     const item = this.#items[this.#inicio];
32
33     // Remove o item do início da fila
34     delete this.#items[this.#inicio];
35
36     // Move o índice do início para o próximo item
37     this.#inicio++;
38
39     // Quando o início e o fim estiverem alinhados, redefine a fila
40     if (this.#inicio === this.#fim) {
41         this.#inicio = 0;
42         this.#fim = 0;
43     }
44
45     return item; // Retorna o item removido
46 }
```



IMPLEMENTAÇÃO MÉTODO FRONT



```
52 // Retorna o primeiro elemento da fila sem removê-lo (peek)
53 front()
54 {
55     // Se a fila estiver vazia, retorna undefined
56     if (this.estaVazia()) {
57         return undefined;
58     }
59     // Retorna o primeiro elemento
60     return this.#items[this.#inicio];
61 }
```



IMPLEMENTAÇÃO MÉTODO LIMPAR

● ● ● Aula 05 - Filas - Fila.js

```
68 // Limpa a fila
69 limpar()
70 {
71     this.#items = {};
72     this.#inicio = 0;
73     this.#fim = 0;
74 }
```



IMPLEMENTAÇÃO DOS MÉTODOS ESTAVAZIA E TAMANHO



Aula 05 - Filas - Fila.js

```
60 // Verifica se a fila está vazia
61 // Verifica se os índices estão iguais
62 estaVazia = () => this.#fim === this.#inicio;
63
64 // Retorna o tamanho da fila
65 // Calcula a diferença entre fim e início
66 tamanho = () => this.#fim - this.#inicio;
```



COMO USAR A CLASSE?



Aula 05 - Filas - app.js

```
1  const Fila = require('./Fila.js');
2
3  const minha_variavel = new Fila();
4
5  minha_variavel.enqueue("Cliente 1");
6  minha_variavel.enqueue("Cliente 2");
7  minha_variavel.enqueue("Cliente 3");
8
9  console.log(minha_variavel.front()); // Saída: "Cliente 1"
10 console.log(minha_variavel.dequeue()); // Saída: "Cliente 1"
11 console.log(minha_variavel.dequeue()); // Saída: "Cliente 2"
12
13 minha_variavel.enqueue("Cliente 4");
14
15 // Saída: 2 (Cliente 3 e Cliente 4 ainda estão na fila)
16 console.log(minha_variavel.tamanho());
17 console.log(minha_variavel.front()); // Saída: "Cliente 3"
```



EXERCÍCIOS SOBRE PILHA

Use a classe implementada acima para resolver os exercícios

EXERCÍCIO 1

- O grande feiticeiro Eldrin vive em uma torre cheia de livros de feitiços. Ele sempre coloca os novos livros no topo da pilha e só pode pegar o que está no topo primeiro.
- Implemente um sistema para ajudá-lo a organizar seus feitiços com as seguintes funcionalidades:
 - Adicionar um novo feitiço no topo da pilha.
 - Remover o feitiço do topo (Eldrin usou e guardou em outro lugar).
 - Ver qual feitiço está no topo sem removê-lo.
 - Verificar se não há mais feitiços na pilha.

EXERCÍCIO 2

- O Capitão Barbarruiva tem um baú cheio de tesouros, mas ele só consegue pegar o último item que colocou.
- Crie um programa para ajudá-lo a:
 - Guardar um novo tesouro no baú.
 - Retirar o último tesouro guardado.
 - Olhar qual é o último tesouro sem pegá-lo.
 - Saber se o baú está vazio.

EXERCÍCIOS SOBRE FILAS

Use a classe implementada acima para resolver os exercícios

EXERCÍCIO 1

- Crie um sistema de atendimento onde clientes entram na fila e são atendidos na ordem de chegada. A cada iteração, um cliente deve ser removido da fila e exibido no console. Novos clientes podem ser adicionados aleatoriamente.
- **Desafio Extra:** Simule diferentes tempos de atendimento para cada cliente, usando `setTimeout()`.

EXERCÍCIO 2

- Simule uma fila de impressão onde diferentes documentos são adicionados à fila. Cada documento deve ter um nome e um tamanho em páginas. Ao processar a fila, exiba no console qual documento está sendo impresso e remova-o da fila após a "impressão".
- **Desafio Extra:** Limite a capacidade da fila (ex: 5 documentos) e exiba uma mensagem quando a fila estiver cheia.

EXERCÍCIO 3

- Imagine um brinquedo que comporta apenas um passageiro por vez. Os visitantes entram na fila e são chamados um por um para embarcar. Crie um sistema que adicione visitantes à fila e simule o embarque e desembarque.
- **Desafio Extra:** Dê um tempo fixo para cada embarque usando `setTimeout()` e adicione um sistema de prioridade para visitantes VIP.

EXERCÍCIO 4

- Simule um restaurante onde os pedidos são adicionados a uma fila e preparados na ordem correta. Cada pedido deve ter um número e um nome (ex: "Pedido 1: Hambúrguer"). À medida que são preparados, os pedidos devem ser removidos da fila.
- **Desafio Extra:** Implemente um sistema onde os pedidos demoram tempos diferentes para serem preparados.

EXERCÍCIO 5

- Simule um caixa de supermercado onde os clientes entram na fila e passam pelo caixa um por um. Cada cliente pode ter um número aleatório de produtos (entre 1 e 10). O tempo de atendimento deve ser proporcional ao número de produtos.
- **Desafio Extra:** Crie várias filas e distribua os clientes entre elas para otimizar o tempo de atendimento.

OBRIGADO!

- Encontre este **material on-line** em:
 - Slides: Google Classroom
- Em caso de **dúvidas**, entre em contato:
 - **Prof. Tiago:** tiago.silva@fatecjahu.edu.br

